RT-IIPF // TECHNICAL REFERENCE

FinGuard-AI: Real-Time Inline Interception Payment Framework

A three-service prototype that scores every UPI payment for fraud *while the money is frozen mid-transaction* — committing or rolling back the ledger based on an AI verdict, in tens of milliseconds.

Stack React · Spring Boot 3 · FastAPI · H2 · scikit-learn

Model Isolation Forest (unsupervised) **Training data** 250,000 UPI transactions

---

01 The problem & the idea
02 System architecture
03 Technology stack
04 Life of a transaction
05 The machine learning core
06 Feature enrichment
07 Database & rollback mechanics
08 Security design decisions
09 The dashboard
10 API reference
11 File map
12 Running & demoing
13 Measured performance
14 Limitations & future work
15 Glossary

[01]

The problem & the idea

Traditional bank fraud systems are **post-hoc**: the payment settles, logs are analyzed minutes or hours later, and if fraud is found the bank tries to claw the money back — often too late. By the time anyone notices, the funds have been withdrawn or forwarded through a chain of mule accounts.

RT-IIPF inverts this. The fraud check happens **inside the payment transaction itself**. When a payment request arrives, the banking switch suspends the executing thread, holds the ledger rows under a database lock, applies the debit and credit *without committing them*, and ships the transaction's metadata to a machine-learning

microservice. Only when the verdict returns does the switch decide: commit the movement, or roll it back atomically as if it never happened — and lock the offending account.

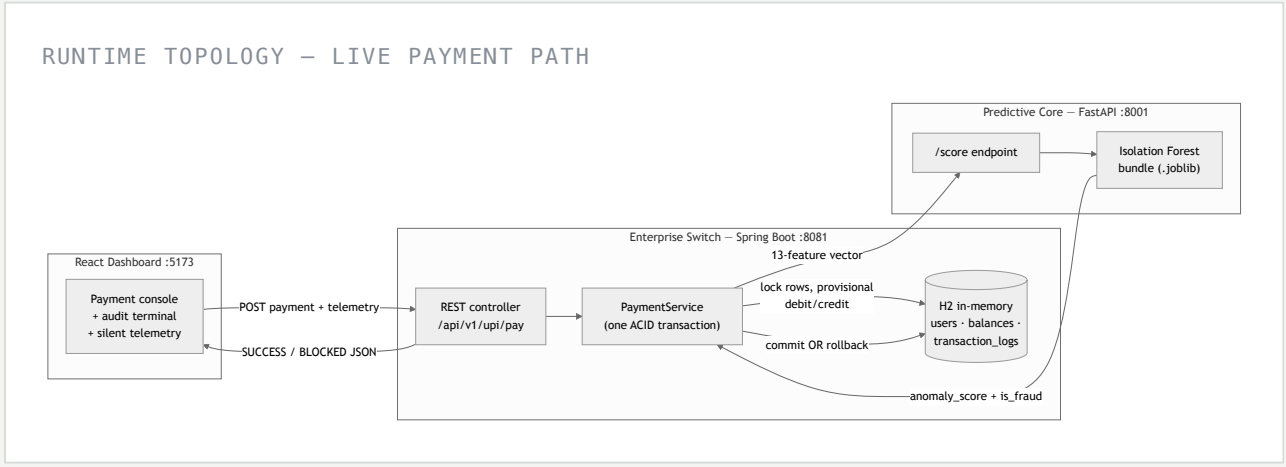
> **[CORE THESIS]**
>
> Fraud detection is moved from *after settlement* to *before settlement*. A blocked transaction never existed in the ledger. There is nothing to claw back, because nothing moved.

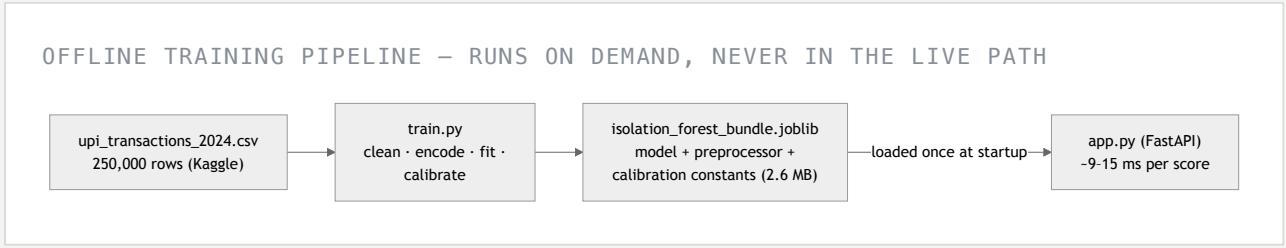
The engineering challenge this creates is latency: while the AI deliberates, real database rows are frozen. The entire round trip — validate, lock, score, verdict, commit-or-rollback — must fit a **sub-50 ms operational window**. That budget shapes every design decision below.

[02]

System architecture

Three independent services, each in its own language and process, speaking plain HTTP + JSON. Nothing shares memory; each piece can be developed, restarted, and scaled alone.





The separation of the two diagrams is itself the point: the heavy 30 MB dataset and minutes-long training run live entirely offline. The live system only ever touches the

frozen 2.6 MB model file, loaded into memory once at startup.

[03]

Technology stack

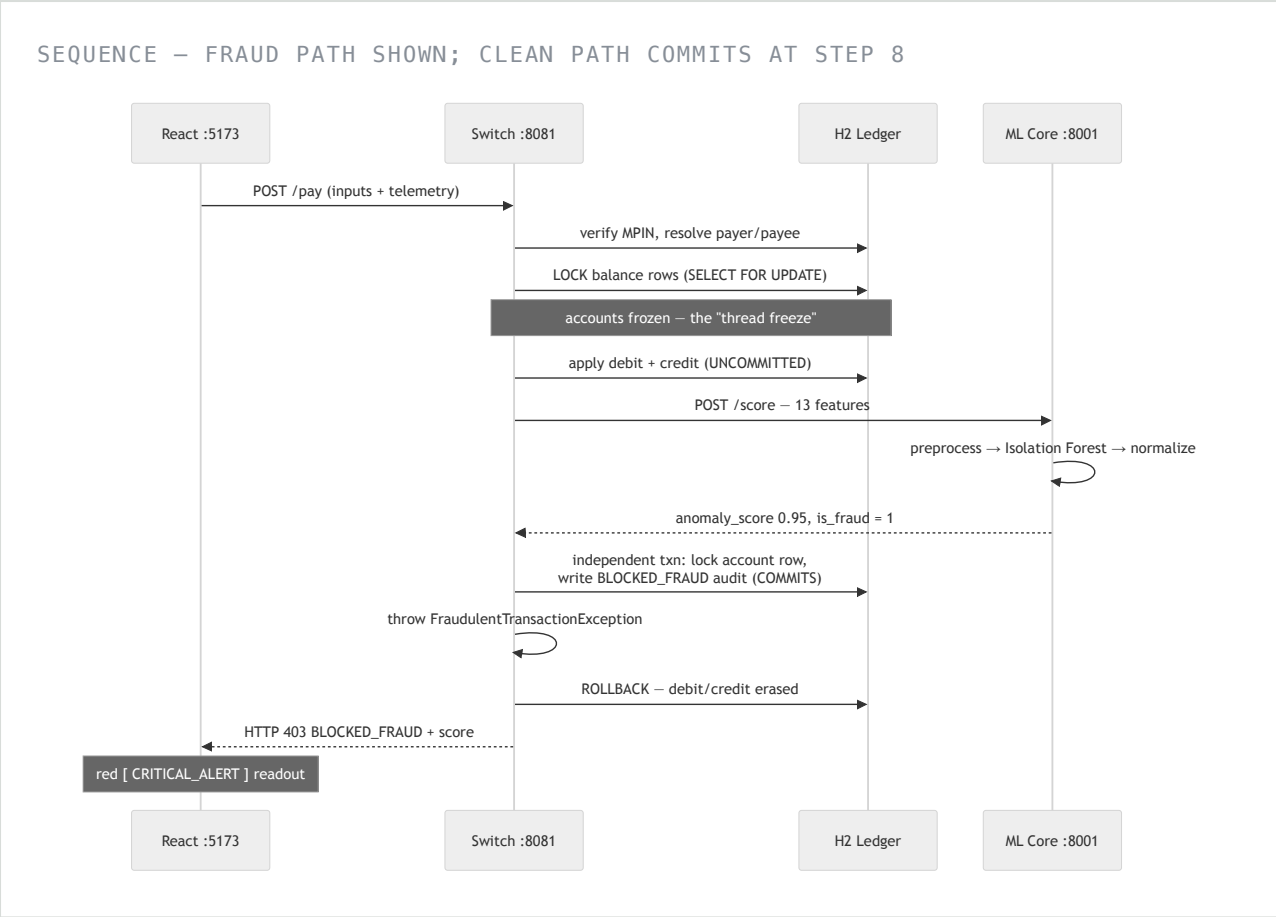
Layer	Technology	Port	Role
Presentation	`React 18 + Vite`	`5173`	Payment console, live audit terminal, service status header
Enterprise switch	`Java 21 · Spring Boot 3.5`	`8081`	Authentication, ledger, ACID commit/rollback, orchestration
Predictive core	`Python 3.14 · FastAPI`	`8001`	Real-time anomaly scoring
Database	`H2 (in-memory) via Spring Data JPA`	—	Three tables: users, balances, transaction_logs
ML tooling	`scikit-learn · pandas · joblib`	—	Offline training and model serialization

> [NOTE — PORTS]
> The original blueprint specified 8080/8000; on the development machine Docker Desktop occupies both, so the switch runs on `8081` and the ML core on `8001`. Both are one-line config changes (`application.yml`, `launch.json`).

[04]

Life of a transaction

Everything below happens in a single request thread, inside one database transaction, in tens of milliseconds.



1. **Capture.** The dashboard collects the six typed fields, silently attaches device telemetry, and posts once.
2. **Authenticate.** The switch resolves both VPAs, rejects locked accounts (HTTP 423), and verifies the MPIN against its SHA-256 hash in constant time (401 on mismatch).
3. **Freeze.** Both balance rows are acquired with `PESSIMISTIC_WRITE` locks — always in ascending user-id order, so two opposing concurrent payments can never deadlock. Insufficient funds → 422.
4. **Provisional movement.** Debit and credit are applied to the persistence context — real changes, but uncommitted. This is the transaction existing in superposition.
5. **The hop.** The thread blocks on a synchronous HTTP call to `/score`, carrying the 13-feature vector. Timeouts are tight (300 ms connect / 1.5 s read) because locks are held the whole time.
6. **Verdict.** The model returns a normalized score in [0,1]. `score ≥ 0.85 → is_fraud = 1`.
7. **Fraud branch.** The account lock and a `BLOCKED_FRAUD` audit row are committed in an *independent* transaction first; then `FraudulentTransactionException` is thrown, rolling the money movement back atomically. The client gets HTTP 403.
8. **Clean branch.** A `SUCCESS` audit row is written and the whole transaction commits. The client gets the new balance and score.

[05]

The machine learning core

Why Isolation Forest, and why unsupervised

An Isolation Forest is an ensemble of 200 randomized decision trees. Each tree repeatedly picks a random feature and a random split point. A typical transaction — sitting deep inside the mass of normal data — takes many splits to isolate. An unusual one gets separated almost immediately. **Average isolation depth, inverted, is the anomaly score.**

The model is unsupervised: it never sees fraud labels during training. It learns the shape of *normal* from 200,000 transactions and flags whatever sits far outside that shape. This matters practically because real fraud labels are scarce, delayed, and stale — and an anomaly detector can catch *novel* fraud patterns no labeled history contains.

Training procedure (train.py)

1. Load the 250K-row CSV; drop post-settlement columns (`transaction_status`) and identifiers — the model may only use what is knowable *before* the money moves.
2. Split 80/20 train/held-out.
3. Fit preprocessing on train only: standard-scale the 3 numerics, one-hot encode the 10 categoricals (`handle_unknown="ignore"` so unseen categories never crash production).
4. Fit the forest (200 trees).
5. **Calibrate the score.** Raw tree scores are min-max normalized to [0,1] — with the ceiling winsorized at the 99.95th percentile of the training distribution rather than the absolute max, so one freak training point cannot compress the whole operational range. Scores beyond the ceiling saturate at 1.0.
6. Serialize model + preprocessor + calibration constants as one `.joblib` bundle.

> [HONESTY CHECK]
>
> This Kaggle dataset's `fraud_flag` labels were verified to be statistically random (ROC-AUC ≈ 0.48 for any detector). The system therefore demonstrates *anomaly detection* — flagging statistically extreme transactions — which is the honest and correct framing for an unsupervised prototype.

Serving (app.py)

FastAPI loads the bundle once at startup and performs a warm-up inference so the first real request doesn't pay one-time lazy-initialization costs (~30 ms). After warm-up, each `/score` call validates the payload against a Pydantic schema, applies the *identical* preprocessor fitted at training time, scores, normalizes with the *identical* constants, and answers in ~9–15 ms. If the model file is missing, the service refuses to start — a scoring node without a brain must never accept traffic.

[06]

Feature enrichment: 6 typed fields → 13 features

The user types six things. The model receives thirteen. The gap is filled by two automatic layers that the user — and therefore an attacker — does not control:

Layer	Source	Features	Trust basis
1 · Typed input	Payment form	`amount, transaction_type, merchant_category`	User-asserted; anomalous lies raise the score
2 · Silent telemetry	`telemetry.js (browser)`	`device_type, network_type (+ device signature, geo-hash)`	Captured automatically at page load; spoofing produces rarer, more suspicious combos
3 · Server enrichment	Database + server clock	`sender/receiver age_group, state, banks; hour_of_day, day_of_week, is_weekend`	Client cannot influence: profiles come from DB rows, time from the switch's own clock (IST)

Enrichment costs essentially nothing: the switch already loads both user rows to check the MPIN and route the money, so the profile features ride along for free — zero extra queries. On the Python side the 13 fields expand via one-hot encoding into the several-dozen-dimensional vector the forest actually consumes.

The VPAs are keys, not features. The MPIN is authentication only and never leaves the switch.

[07]

Database & rollback mechanics

Three tables, managed by JPA/Hibernate against in-memory H2:

Table	Contents	Notable columns
`users`	Account holders	`vpa (unique), mpin_hash, age_group, state, bank, account_locked`
`balances`	The ledger — one row per user	`amount_inr DECIMAL(15,2) — money is BigDecimal, never floats`
`transaction_logs`	Immutable audit trail of every attempt	`txn_ref, anomaly_score, fraud_verdict, status, processing_ms`

The three transactional tricks

1 — The freeze is a row lock. "Suspending the thread" is implemented as `SELECT … FOR UPDATE` (`PESSIMISTIC_WRITE`) on both balance rows. No concurrent transaction can touch those balances until the verdict resolves. Locks are always acquired in ascending user-id order — the classic deadlock-prevention discipline.

2 — The rollback is an exception. The debit/credit are applied inside the open transaction but never explicitly committed. On a fraud verdict the service throws `FraudulentTransactionException` ; Spring's `@Transactional` machinery marks the transaction rollback-only, and every uncommitted change evaporates atomically. The commit path is simply *not throwing*.

3 — The audit survives the rollback. A paradox: the rollback must erase the money movement, but the bank must remember the blocked attempt and keep the account locked. Solution: `AccountSecurityService` runs with `REQUIRES_NEW` propagation — it suspends the payment transaction, commits the account lock and the `BLOCKED_FRAUD` audit row in its own independent transaction, and only then does the payment transaction roll back. Two outcomes, two transactions, both correct.

Security design decisions

Decision	Implementation	Why
Fail closed	ML core down / timeout → HTTP 503, payment rejected	A payment is never waved through unscored; availability is sacrificed before safety
No plaintext MPINs	SHA-256(vpa:mpin), constant-time comparison	DB theft doesn't yield PINs; timing attacks blunted (production would use a KDF like bcrypt)
Server-derived time	hour/day/weekend from the switch clock, not the client	A fraudster cannot pretend to transact at a normal hour
DB-derived profiles	Age/state/bank resolved from user rows, never accepted from the client	A hostile client cannot spoof its own risk profile
Lock on verdict	`account_locked` flag set in the independent transaction	A flagged account cannot immediately retry; survives the rollback
Tight client timeouts	300 ms connect / 1.5 s read on the ML hop	Row locks are held during the hop; a hung ML node must not freeze accounts indefinitely
No stack-trace leakage	Global exception handler maps every failure to structured JSON	Stable machine-readable `code` fields; internals stay internal

[09]

The dashboard (RT-IIPF Core Switch Monitor)

A deliberately industrial, terminal-styled operations console — two columns on desktop:

- **Status bar** — polls both backends every 4 s: `FASTAPI CORE: CONNECTED (:8001)` flips red within seconds of the service dying. The states are live, not decorative.
- **Left: payment console** — origin account picker with live balance/lock strip, destination VPA, amount, type, MCC category, MPIN. During the hop, the button reads `>> SCORING INLINE — THREAD HELD <<`. Verdicts render as stark readouts: green `[ COMMIT ]`, red `[ CRITICAL_ALERT ]`, amber `[ REJECTED ]`.
- **Right: audit terminal** — a black `tail -f` viewport over `transaction_logs`, one raw line per attempt:

```
2026-07-17 14:53:17 [ OK ] TXN: 519c-2b | arjun@fg -> grocery@fg | AMT: ₹100 | SCORE: 0.645 | LA
2026-07-18 11:27:44 [ CRITICAL_ALERT ] TXN: f191-20 | priya@fg -> grocery@fg | AMT: ₹60,000 | SC
```

[10]

API reference

Endpoint	Method	Purpose	Outcomes
/api/v1/upi/pay	POST	Execute a payment (switch :8081)	200 SUCCESS 403 BLOCKED_FRAUD 401 · 422 · 423 · 400
/api/v1/upi/account/{vpa}	GET	Balance + lock state	200 / 404
/api/v1/upi/transactions	GET	Last 20 audit rows (feeds the terminal)	200
/score	POST	13 features → verdict (ML core :8001)	200 / 422 / 503
/health	GET	ML liveness + model metadata	200 / 503
/h2-console	GET	Browser SQL console (JDBC `jdbc:h2:mem:finguard`, user `sa`)	—

Error bodies are uniform JSON: `{"timestamp", "status", "message"}` with a stable machine-readable `status` code (`INVALID_MPIN` , `INSUFFICIENT_FUNDS` , `ACCOUNT_LOCKED` , `SCORING_UNAVAILABLE` …).

[11]

File map

```
ieee/
├── upi_transactions_2024.csv        ← 250K-row Kaggle training data (offline only)
│
├── ml-service/                      ── PYTHON: THE BRAIN (:8001)
│   ├── train.py                     ← offline trainer: CSV → fitted model
│   ├── models/
│   │   └── isolation_forest_bundle.joblib  ← the frozen brain (2.6 MB)
│   ├── app.py                       ← FastAPI: /score + /health, loads bundle at boot
│   └── requirements.txt
│
├── switch-service/                  ── JAVA: THE BANK (:8081)
│   ├── pom.xml                      ← Maven build + dependencies
│   └── src/main/
│       ├── java/com/finguard/switchsvc/
│       │   ├── FinGuardSwitchApplication.java
│       │   ├── entity/              ← User · Balance · TransactionLog (the 3 tables)
│       │   ├── repository/          ← JPA data access; the FOR-UPDATE lock lives here
│       │   ├── service/             ← PaymentService (the pipeline) · FraudScoringClient
│       │   │                          · AccountSecurityService (REQUIRES_NEW writes)
│       │   ├── controller/          ← REST endpoints
│       │   ├── dto/                 ← request/response shapes, validation rules
│       │   ├── exception/           ← FraudulentTransactionException + error mapping
│       │   └── config/              ← ML http client · demo data seeder
│       └── resources/application.yml ← ports, H2, ML base-url, timeouts
│
├── upi-client/                      ── REACT: THE SCREEN (:5173)
│   ├── src/App.jsx                  ← entire UI: console + terminal + status bar
│   ├── src/App.css                  ← industrial terminal skin
│   ├── src/api.js                   ← fetch calls to both backends
│   ├── src/telemetry.js             ← silent device fingerprinting
│   └── vite.config.js · index.html · package.json
│
└── .claude/launch.json              ← dev-server launch config (all three services)
```

Running & demoing

Start the stack (three terminals, in this order)

```
# 1 — ML core (the switch fails closed without it)
cd ~/ieee/ml-service && source .venv/bin/activate
uvicorn app:app --host 127.0.0.1 --port 8001

# 2 — Switch
export JAVA_HOME=/opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home
$JAVA_HOME/bin/java -jar ~/ieee/switch-service/target/finguard-switch-1.0.0.jar

# 3 — Dashboard
cd ~/ieee/upi-client && npm run dev
```

Then open `http://localhost:5173`.

Demo accounts (seeded on every switch start)

VPA	Name	MPIN	Balance
`arjun@fgbank`	Arjun Mehta	4321	₹58,000
`priya@fgbank`	Priya Nair	9876	₹1,25,000
`ravi@fgbank`	Ravi Kumar	2468	₹9,000
`grocery@fgpay`	Grocery Mart (merchant)	—	₹3,10,000

The three-act demo script

1. `GREEN` Arjun → `grocery@fgpay`, ₹250, P2M/Grocery, MPIN 4321 → `[ COMMIT ]`, score ≈ 0.1, balances move.
2. `RED` Priya → `grocery@fgpay`, **₹60,000, Recharge/Other**, MPIN 9876 → `[ CRITICAL_ALERT ]`, score ≈ 0.95+, rollback, Priya locked. Show her balance: unchanged. Show the H2 console: the BLOCKED_FRAUD audit row exists while the ledger is untouched.
3. `FAIL-CLOSED` Kill the Python process; watch the header flip to `FASTAPI CORE: OFFLINE`; attempt any payment → 503 rejection, zero money moved. Restart it; payments resume.

> `[ RESET ]`
>
> H2 is in-memory: restarting the switch is a full factory reset — fresh balances, locks cleared, empty audit trail.

`[ 13 ]`

Measured performance

Measurement	Value
ML inference hop (warm)	`9–15 ms`
End-to-end switch processing (auth → verdict → commit)	`20–50 ms`
Model load at ML startup	`~1 s (off the hot path)`
Warm-up inference (kills first-request lazy-init cost)	`~25 ms, once`

Real scored examples from this system, showing the joint-distribution effect — the same typed input scores differently as context changes:

Transaction	Score	Verdict
₹250 P2M Grocery, Android/4G, midday	`0.092`	`[ OK ]`
Same ₹250 — but from a desktop browser (Web/3G)	`0.639`	`[ OK ]` elevated
₹42,000 alone, midday weekday context	`0.801`	`[ OK ]` — amount alone can't trip it
₹60,000 Recharge/Other, Web/4G	`0.948`	`[ CRITICAL_ALERT ]`
₹60,000 Recharge/Other, Web/3G	`0.989`	`[ CRITICAL_ALERT ]`

`[ 14 ]`

Limitations & future work

- **Population-level, not personal.** The model knows what's abnormal for *everyone*, not what's normal for *this user*. A wealthy user's routine ₹50K transfer and a

compromised account's first ₹50K drain look identical. Fix: per-account rolling baselines (deviation from own mean amount, usual hours, known payees, known devices).

- **No velocity features.** A rapid drain via many small transfers is invisible — each one looks normal alone. Fix: transactions-per-hour / amount-per-day features from one indexed query over `transaction_logs`, plus a retrain.
- **Binary response.** One threshold, hard block, permanent lock. Production would grade the response: 0.85–0.95 → step-up authentication (OTP/biometric); >0.95 → block; lock released by support workflow, not never.
- **H2 volatility.** Every switch restart wipes the ledger. Fine for demos (free reset); one config line moves H2 to file-backed persistence.
- **Prototype auth.** SHA-256 MPIN hashing → bcrypt/argon2; no session tokens or TLS; CORS pinned to localhost.
- **Dataset labels are noise.** Supervised evaluation is impossible on this data; a labeled dataset (or synthetic injected fraud) would let precision/recall be measured meaningfully.

Glossary

UPI	Unified Payments Interface — India's real-time interbank payment system
VPA	Virtual Payment Address — email-style money handle (`name@bank`) that hides real account numbers
MPIN	Mobile PIN — the 4–6 digit secret authorizing a UPI payment
MCC	Merchant Category Code — classifies what a merchant sells (Grocery, Fuel…)
Switch	Banking-industry term for the system that routes and settles payment messages
Isolation Forest	Unsupervised anomaly detector; outliers isolate in fewer random splits than normal points
Contamination	The forest's prior for what fraction of data is anomalous (~0.2% here)
.joblib	Python binary serialization format used to freeze the trained model to disk
Pydantic BaseModel	Request-validation template in FastAPI — a *data* model, not an ML model
Pessimistic lock	`SELECT … FOR UPDATE` — blocks concurrent access to a row until the transaction ends
REQUIRES_NEW	Spring propagation: run this write in its own transaction so it survives the caller's rollback
Fail-closed	When the safety check is unavailable, refuse the action — never default to allowing it
H2	Lightweight in-memory SQL database embedded in the Java service

---